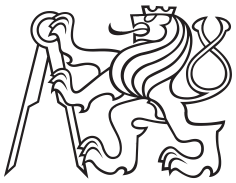
Master Thesis



Czech Technical University in Prague

F3

Faculty of Electrical Engineering
Department of Computer Science

Neurogenesis of Neural Networks for 3d Nesting

Filip Špidla

Supervisor: Ing. Mgr. Ladislava Smítková Janků, Ph.D.
Field of study: Artificial Intelligence
August 2025

Acknowledgements

Děkuji ČVUT, že mi je tak dobrou *alma mater*.

Declaration

Prohlašuji, že jsem předloženou práci vypracoval samostatně, a že jsem uvedl veškerou použitou literaturu.

V Praze, 22. August 2025

Abstract

This thesis proposes a learning-guided system for 3D nesting and transport of convex items that optimizes utilization under non-overlap, transport and container-bound constraints. It aims to explore use of neurogenesis in order to appraise its viability for this class of problems.

It proposes state exploration using heuristics to enumerate feasible loading plans combined with neural scoring models and architectures to rank candidate solutions. Model architectures are evolved via neurogenesis.

Items, containers, and candidates are represented with compact local and global features capturing size, capacity usage, and transport attributes. We survey the state of the art and build a reproducible benchmark. Evaluation reports utilization, bin count, cost, and runtime.

Keywords: nesting, neurogenesis, artificial intelligence, bin packing

Supervisor: Ing. Mgr. Ladislava Smítková Janků, Ph.D.

Abstrakt

Tato práce navrhuje systém pro 3D nesting konvexních objektů řízený učením, který optimalizuje využití prostoru při dodržení vymezení jako je nepřekrývání, přepravních požadavků a atributy kontejneru. Cílem je prozkoumat použití neurogeneze a posoudit její vhodnost pro tuto třídu úloh.

Navrhuje prozkoumávání stavového prostoru pomocí heuristik, které enumerují proveditelné plány nakládky, v kombinaci s neuronovými skórovacími modely a architekturami pro řazení kandidátních řešení. Architektury modelů jsou vyvíjeny evolučně pomocí neurogeneze.

Položky, kontejnery a kandidáti jsou reprezentováni kompaktními lokálními i globálními vlastnostmi zachycujícími rozměry, využití kapacity a přepravní atributy. Posuzuje současné state-of-the-art řešení této třídy problému a navrhuje reprodukovatelný benchmark. Vyhodnocení bude uvádět míru využití, počet binů, náklady a dobu běhu.

Klíčová slova: polohování, neurogeneze, umělá inteligence, bin packing

Překlad názvu: Neurogeneze neuronových sítí pro 3D nesting

Contents

Figures

Tables

Chapter 1

Introduction

The 3d bin-packing problem is a common optimization challenge in which items must be placed into a limited number of bins while maximizing space utilization. As the problem is NP-hard, computing an optimal solution with deterministic algorithms quickly becomes infeasible as the number of items grows.

Classical heuristic methods can be fast and effective in specific regimes, but their decision rules are difficult to tune and often fail to transfer when constraint sets or item distributions change.

This thesis explores the idea that neural networks, designed through neuro-genesis—the evolutionary construction and tuning of neural architectures-can help address these challenges. The proposed approach combines state-of-the-art geometric heuristics to generate feasible placement candidates, while an evolved neural network evaluates these candidates and selects the next action. Because the underlying nesting function is highly complex and varies with different constraint combinations, evolutionary search provides a promising way to discover neural architectures that can adapt to this variability.

Chapter 2

Problem analysis

This chapter serves as an introduction to concepts this work addresses and outlines scope of this work.

It formally defines the 3D bin packing problem. We then introduce neurogenesis as the core methodology for evolving neural network architectures in this discrete optimization setting.

In addition, the chapter provides an overview of reinforcement learning, and deep reinforcement learning in particular, as these components forms important theoretical foundation for the method developed in this work.

2.1 Problem definition

2.1.1 Bin packing

Cutting and packing problems can be broadly summarized with following description.
Given are two sets of elements, namely

- a set of large objects (input, supply) and

- a set of small items (output, demand),

which are defined in some number of geometric dimensions. Select some or all small items, group them into one or more subsets and assign each of the resulting subsets to one of the large objects such that the geometric condition holds, i.e. the small items of each subset have to be laid out on the corresponding large object such that

- all small items of the subset lie entirely within the large object and
- the small items do not overlap,

and a given (single-dimensional or multi-dimensional) objective function is optimised.[WHS07]

Definition 2.1 (3D bin packing)**.** Let $I = \{1, \dots, n\}$ be items with sizes $(w_i, h_i, d_i) \in \mathbb{R}^3_{>0}$ and per-item sets of allowed orthogonal rotations $R_i \subseteq S_3$. Bins are identical axis-aligned boxes of size $(W, H, D) \in \mathbb{R}^3_{>0}$.

A *packing* assigns each item i a bin index $k \in \{1, \dots, K\}$, an orientation $r_i \in R_i$, and a position $\mathbf{x}_i = (x_i, y_i, z_i) \in \mathbb{R}^3_{\geq 0}$ such that:

(in-bounds) $0 \leq x_i \leq W - w_i^{r_i}, \;\; 0 \leq y_i \leq H - h_i^{r_i}, \;\; 0 \leq z_i \leq D - d_i^{r_i},$

(non-overlap) $\forall i \neq j$ in the same bin k : $(x_i{+}w_i^{r_i} \leq x_j) \;\vee\; (x_j{+}w_j^{r_j} \leq x_i) \;\vee$
$(y_i{+}h_i^{r_i} \leq y_j) \;\vee\; (y_j{+}h_j^{r_j} \leq y_i) \;\vee (z_i{+}d_i^{r_i} \leq z_j) \;\vee\; (z_j{+}d_j^{r_j} \leq z_i),$

where $(w_i^{r_i}, h_i^{r_i}, d_i^{r_i})$ denotes the dimensions of item i permuted by r_i.

Objectives. Either (i) *min-bins*: pack all items using the minimum number K of bins; or (ii) *max-utilization*: given a bin budget K, pack a subset $J \subseteq I$ to maximize $\sum_{i \in J} w_i^{r_i} h_i^{r_i} d_i^{r_i}$ (equivalently, utilization per bin).

Additional constraints. Let $\mathcal{C}$ denote application-specific constraints (e.g., weight limits, stability/center-of-mass, accessibility/LIFO, orientation allowances, compatibility/segregation). A packing is $\mathcal{C}$*-feasible* if it satisfies the geometric conditions above and all $c \in \mathcal{C}$.

We target 3D bin packing with multiple bins: given a set of rectangular boxes (convex items) with orthogonal rotations, pack them efficiently into minimum number of identical containers[WHS07]

2.1.2 Constraints

Cutting and packing problems are further defined by their constraints. Beyond the basic geometric feasibility (in-bounds, non-overlap), the literature further mentions

Orientation item-specific allowed orientations, fragile handling, stackability classes[dNAJ21]

Global capacity / weight per-bin weight limits, sometimes combined with volumetric limits[dNAJ21]

Stacking / load-bearing limits on vertical support to avoid damaging items, may depend on item type or allowable stack height.[GTMP22]

Stability & center of mass static equilibrium, and even dynamic stability formulations to avoid tip-over during handling or motion[GOGL16]

Accessibility LIFO or door-side accessibility so earlier-stop items are not blocked. Some works use soft penalization rather than hard constraints[BTC23]

2.2 Neurogenesis

Neurogenesis or neuroevolution is a practice of evolving the architecture and hyperparameters of given neural scoring models with a genetic algorithm. Classical neurogenesis like NEAT established that evolving topology alongside weights is effective.[LSX+20] Latter techniques such as Efficient Neural Architecture Search, shortly ENAS, focuses mostly on development of deep neural networks.[RMS+17]
Artificial neural network architecture design is a nontrivial and time-consuming task that often requires a high level of human expertise. Neurogenesis serves to automate the design of NN architectures and has proven to be successful in automatically finding NN architectures that outperform those manually designed by human experts.[BB24]

2.2.1 Principles

In principle, neural network design is viewed as a population-based search over both structure (topology/connection patterns) and parameters. Individuals

encode candidate architectures and hyperparameters. Each is initiated, evaluated by an outcome-based fitness function. The target fitness is to be improved via actions of genetic algorithm of selection, crossover, mutation, and elitism.

2.2.2 Search space and genome encoding

We encode a candidate network as a vector of genes. Compact search space keeps evaluation times manageable. Some of the examples include following:[WSS+23]

- **Depth/width:** number of layers (e.g., 2–6); hidden sizes (e.g., 64/128/256/512).
- **Activations & normalization:** {ReLU, GELU, SiLU} and {none, LayerNorm, BatchNorm}.
- **Skip/residuals:** on/off per stage.
- **Context aggregation (optional):** {none, local MLP pooling, small GNN over kNN}.
- **Feature subset:** bitmask/index selecting which engineered features are used.
- **Regularization/training:** dropout rate, weight decay, learning-rate schedule.

Gene	Example domain
Depth	$\{2, 3, 4, 5, 6\}$
Hidden width	$\{64, 128, 256, 512\}$
Activation	{ReLU, GELU, SiLU}
Normalization	{none, LayerNorm, BatchNorm}
Residuals	{off, on}
Context aggregation	{none, MLP pooling, small GNN}
Feature subset	index / bitmask over features
Dropout	$[0, 0.3]$
Weight decay	$[0, 10^{-3}]$
LR schedule	{cosine, step, warmup}

Table 2.1: Genome (architecture & training) encoding for the scoring model.

2.2.3 Fitness design

Fitness is computed by outcome: each individual (architecture + hyperparameters) is instantiated, run on a batch of task instances, and mapped to a scalar for elitism. A simple scalarization is

$$\text{Fit} \;=\; -\,\text{TaskCost} \;-\; \lambda_{\text{viol}} \cdot \text{Violations} \;+\; \lambda_{Q} \cdot \text{Quality} \;-\; \lambda_{t} \cdot \text{Runtime},$$

with weights set to reflect priorities. Alternatively, multiple objectives can be kept in a Pareto set (non-dominated sorting).[EMH19a] To reduce variance and cost, average fitness over small instance batches and use multi-fidelity evaluation (cheap proxies early; full fidelity for shortlisted elites).[WSS+23]

2.2.4 Evolutionary operators

Evolutionary operators in neurogenesis follow standard genetic algorithm, specialized for neural networks by using a genome that encodes architecture and hyperparameters. The operators below are adapted to these NN-specific genomes.

Initialization. Sample genomes uniformly; optionally seed a few hand-designed baselines.

Selection. Tournament or rank-based selection; for distance vs. runtime trade-offs, use non-dominated sorting (NSGA-II style).

Crossover. One-point or uniform crossover for discrete genes; arithmetic/BLX-α blending for continuous ones.[TK01]

Mutation. Small edits - flip activation/normalization, ± 1 layer, resample feature subset, jitter dropout/weight decay, optionally self-adaptive mutation rates.[LSX+20]

Elitism, replacement. Keep the top E elites; fill the rest with best offspring; optionally add age/diversity pressure.

2.3 Reinforcement learning

Reinforcement learning (RL) is a framework for sequential decision-making in which an agent interacts with an environment in discrete time steps, observes its state, selects actions, and receives scalar rewards. The agent's objective is to learn a policy that maximizes the expected cumulative reward over time.[SB18] Formally, this interaction is often modelled as a Markov decision process (MDP) with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel $P(s' \mid s, a)$, and reward function $r(s, a)$.

At each time step t, the agent observes a state $s_t \in \mathcal{S}$, chooses an action $a_t \in \mathcal{A}$, and then receives a reward $r_t = r(s_t, a_t)$ while the environment transitions to a new state $s_{t+1} \sim P(\cdot \mid s_t, a_t)$. The goal is to find a policy $\pi(a \mid s)$ that maximizes the expected return

$$\mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right],$$

where $\gamma \in [0, 1)$ is a discount factor and T is the episode horizon.[SB18]

2.3.1 Learning for 3D nesting

For 3D nesting with realistic industrial constraints, large labeled datasets of "optimal" or expert packings are scarce. Existing datasets and benchmarks typically provide item sets and container specifications, but not sequences of placements or even universally accepted best layouts. Moreover, even if such datasets were available, supervised learning would be limited to at best reproducing the behavior implicit in the labels, rather than exploring policies that may improve upon existing heuristics.

Objective of this work is not merely to imitate current bin-packing heuristics, but to search for the best nesting strategies given constraints. Reinforcement learning is particularly well-suited to this setting for several reasons:

2.3.2 Deep Reinforcement Learning

Classical reinforcement learning algorithms such as tabular Q-learning and policy iteration maintain explicit value tables with one entry per state-action

pair. This approach becomes computationally intractable when the state space exceeds approximately 10^6 states due to memory constraints and the sample complexity required to visit each state sufficiently often. For example, a 10×10 grayscale image has $256^{100} \approx 10^{240}$ possible states—far beyond the reach of tabular methods.

Deep reinforcement learning addresses this scalability barrier by using neural networks as function approximators: instead of storing $Q(s, a)$ in a table, we represent it as $Q(s, a; \theta)$ where θ are the parameters of a deep neural network. This enables generalization across similar states, dramatically improving practical sample efficiency compared to tabular methods. Empirical results demonstrate that neural function approximation can successfully scale to problems with state spaces orders of magnitude larger than feasible for tabular approaches.[MKS+13]

- **Learning from interaction instead of labels.** The packing environment can be simulated, so an agent can generate its own experience by repeatedly constructing packings and observing rewards that encode utilization, constraint violations, and runtime. No pre-labeled dataset of "correct" actions is required.[SB18]
- **Direct optimization of task-specific objectives.** The reward function can combine multiple criteria (e.g., fill ratio, stability, accessibility penalties), allowing the agent to directly optimize the same objectives that define solution quality in practice.[MSIB20]
- **Potential to outperform hand-crafted heuristics.** Since the agent is free to explore the space of decision policies, it is not constrained to mimic any single heuristic and should theoretically be able to beat the existing heuristics.

2.4 Neural Scoring Models

While heuristics can efficiently enumerate feasible candidate placements using extreme points or empty maximal spaces, the core difficulty in constrained 3D nesting is **selecting among many feasible candidates** under heterogeneous objectives. Classical approaches rely on hand-crafted scoring rules that prioritize specific metrics (e.g., minimize residual void, maximize support area), but these rules are brittle and fail to generalize when constraint sets or item distributions change.

The role of learned scoring. A neural scoring model replaces fixed tie-breakers with a parameterized function that learns to rank placement candidates from outcome-based feedback. At each packing step, the model evaluates candidates using:

- **Item features**: dimensions, weight, orientation constraints, fragility class
- **Placement features**: position, residual void, support surface, clearance
- **Global state**: current utilization, bin capacity, item count

The network outputs a scalar score; the highest-scoring feasible placement is selected. This decomposition—deterministic candidate generation followed by learned selection—preserves feasibility guarantees while enabling adaptation to complex, multi-objective fitness landscapes.

Why neural networks? The discrete, non-differentiable nature of placement decisions precludes gradient-based optimization through the packing process. Neural scoring models accommodate this by:

1. Operating as **black-box evaluators** within a constructive heuristic
2. Accepting compact, engineered feature vectors rather than raw geometry
3. Enabling outcome-based training via evolutionary search (Section 2.2)

This formulation treats the packing pipeline as a Markov decision process where the policy (scoring model) is evolved rather than trained by gradients, fitting naturally into the neurogenesis framework while maintaining computational tractability during inference.

2.5 Problem Characteristics

Given this analysis, we arrive at several challenges and particularities that will have to be addressed by proposed solution:

- **Sparse Reward Structure**: RL rewards are naturally sparse in sequential packing - only terminal/infrequent feedback
- **Large and Fragmented Search Space**: Combinatorial explosion of positions, EMS fragmentation, exponential branching
- **Architectural Compatibility Requirements**: Neurogenesis operates within structured ANN architectures - genome must encode valid networks
- **Computational Constraints**: Fast evaluation required for larger instances and GA population assessments
- **Absence of Labeled Data**: Fast evaluation required for larger instances and GA population assessments
- **Geometric Invariances**: Input equivariance properties - rotational symmetries, position invariances

2.6 Scope of This Work

This work combines neural architecture search with deep reinforcement learning for the 3D bin packing problem. We aim to demonstrate that genetic algorithms can automatically discover neural network architectures suited to constraint-specific packing scenarios without manual hyperparameter tuning.

2.6.1 Problem Scope

We address offline 3D bin packing where all items are known in advance. Items are rectangular boxes with axis-aligned orientations, gravity, multiple bins and constraints may be present.

We do *not* address online/dynamic packing (items arriving in real-time), vehicle routing integration, non-convex items or arbitrary 3D rotations.

2.6.2 Technical Description

The system integrates three key components:

Bin packing. Thesis needs to create an effective bin packing environment for 3D cuboid geometry enables gravity simulation, spatial feature extraction and supports chosen constraints. Upon it a heuristic that generates geometrically feasible placement positions.

Neural Scoring Model. Neural network evaluates candidate placements. The architecture (layer depth, hidden dimensions, attention type, activation functions, dropout) is determined by evolutionary search rather than manual design.

Genetic Architecture Search. A genetic algorithm evolves neural network architectures in order to facilitate improved learning capabilities. Neurogenesis operators apply over multiple generations. Fitness combines packing utilization, bin count, and model complexity.

2.6.3 Evaluation Strategy

We evaluate on available public benchmark datasets. The goal is to demonstrate that:

1. Evolved architectures outperform random architectural choices
2. The system can learn effective placement strategies under multiple constraint types
3. Genetic search discovers problem-appropriate inductive biases (e.g., convolutional spatial reasoning, attention over actions)

Chapter 3

State-of-the-art solutions

After defining the problem of three-dimensional bin packing and motivating the need for an efficient, adaptive optimization solution, this chapter reviews the current state-of-the-art in related methods. The goal is to summarize the existing approaches and appraise them based on problem characteristics in section 2.5. We will further highlight the research gaps that need to be patched up in chapter 4.

This chapter is divided into Three main sections. The first section explores nesting, focusing on heuristic methods developed for the 3D bin packing problem. The second section focuses on neural networks. Third section is about neurogenesis and related methods of efficient neural architecture search. Many papers intersect in some parts or use concepts talked about in other sections, they are assigned into sections based on their primary contribution to this work. Lastly, we shall mention which problem of this task in particular are currently lacking in state-of-the-art and will need to be figured out on its own in this work.

3.1 Nesting

This section categorizes solution strategies for the class of nesting problems. Individual proposed approaches can be broadly split by several characteristics which play a crucial role in determining how algorithm may approach nesting. They provide a handy overview when evaluating an approach's usefulness in

particular scenarios, so I shall list several of them at the beginning of this section.
Rest of the section outlines current state-of-the-art algorithms for class of nesting based problems, along with papers focusing on certain important components such as candidate placements in particular.

Information model (offline vs. online). In the offline setting, the entire item set is known, enabling pre-sorting and deeper look-ahead; online packing processes items as they arrive and must make irrevocable decisions. State-of-the-art strategies in both regimes share a constructive skeleton (candidate generation → selection) but differ in how they balance exploration, robustness, and speed; online variants typically adopt myopic but well-regularized scoring rules and occasional re-packing windows [ARCO22].

Space representation (rasterization vs. geometry-first). Raster/voxel models offer simple, parallelizable collision checks and convenient stability features at the cost of resolution dependence. Geometry-first models rely on exact or approximate intersection tests for convex bodies, oriented bounding boxes, separating-axis or GJK/EPA checks, and precomputed structures (no-fit relations in 2D; extreme points or empty maximal spaces in 3D). Hybrid designs are common: geometry for candidate generation and feasibility, rasterized descriptors for local voids/support [ARCO22].

Constructive heuristics. Constructive backbones couple item ordering (e.g., by size, shape complexity, or "hardness") with candidate evaluation rules (minimize residual void, maximize support, minimize overhang, respect separations and orientation sets). These rules deliver fast feasible layouts and provide consistent starting points for improvement phases [ARCO22].

Neural scoring and learned proposals. Neural components most often rerank EP/EMS candidates instead of proposing arbitrary poses. Feature sets typically mix void/waste metrics, clearances, support/contact estimates, and simple surrogates for load distribution or access. Lightweight models (MLPs, attention over placed items) can replace hand-crafted tie-breakers without changing the surrounding heuristic or matheuristic scaffold [ARCO22].

Local improvement. After a feasible placement, local refinements (swap, rotate within allowed sets, reinsert, neighborhood re-pack) and compaction steps reduce fragmentation. Physics-inspired or projection-based moves

gently compress the layout; when overlaps appear, constraints are restored by expanding or re-projecting to the feasible set. Such micro-optimizations target the highest-void regions and are effective under strict time budgets [ARCO22].

Metaheuristics, matheuristics. GRASP, VNS, tabu, and LNS frameworks explore item orderings and partial re-packs; matheuristics embed exact subproblems (e.g., orientation/placement pricing, stability/separation checks) to sharpen feasibility and improve bounds. This hybridization remains the most common route to high fill ratios with realistic industrial constraints [ARCO22].

3.1.1 Candidate placement heuristics

Modern 3D solvers avoid global pose search by enumerating high-quality placements at the evolving packing frontier. Two families dominate: (i) *extreme points*, induced by faces/edges/vertices of the container and already placed items, and (ii) *empty maximal spaces*, maintained as items are inserted. Both admit quick filters for container bounds, non-overlap, minimal support, separations, and admissible orientations [ARCO22].

Extreme points. Crainic et al. (2007) introduces concept of extreme points and propose new extreme point based online, geometric heuristic for packing items inside 3D container. [CPT08]

Given a packing and the list `3DEPL` of Extreme Points (EPs) defined by the items already placed, Algorithm 1 computes the new EPs that must be added to the list after placing item k at position (x_k, y_k, z_k). The main idea is as follows:

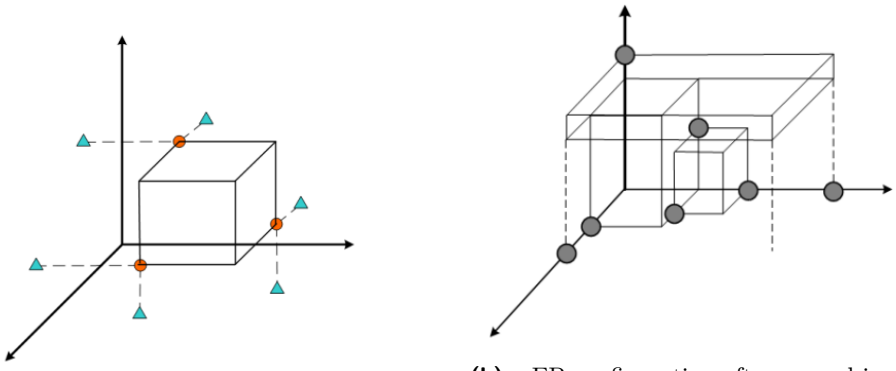
- **Empty container:** If the container is empty, item k is placed at $(0, 0, 0)$, which generates the following three EPs:

$$(w_k,\, 0,\, 0), \quad (0,\, d_k,\, 0), \quad (0,\, 0,\, h_k).$$

- **Non-empty container:** When item k is placed at (x_k, y_k, z_k), new EPs are generated by projecting:
  - the point $(x_k + w_k,\ y_k,\ z_k)$ along the Y and Z axes,

- the point $(x_k,\ y_k + d_k,\ z_k)$ along the X and Z axes,
- the point $(x_k,\ y_k,\ z_k + h_k)$ along the X and Y axes.

- **Projection rule:** Each projected point is moved until it touches either another item or the container wall in the respective direction. If more than one item is intersected along the projection direction, the algorithm chooses the nearest one.



(a) : EP projections from one insertion.

(b) : EP configuration after several insertions.

Figure 3.1: Illustration of Extreme Point generation and accumulation[ARCO22]

Algorithm 1 Generation of new Extreme Points after placing item k

Require: Current packing; list of EPs `3DEPL`; item k with size (w_k, d_k, h_k) placed at (x_k, y_k, z_k)
Ensure: Updated list `3DEPL`

if container is empty **then**
    place item k at $(0, 0, 0)$
    add EPs: $(w_k, 0, 0),\ (0, d_k, 0),\ (0, 0, h_k)$
    return `3DEPL`
end if
define candidate points:

$$p_1 = (x_k + w_k,\ y_k,\ z_k), \quad p_2 = (x_k,\ y_k + d_k,\ z_k), \quad p_3 = (x_k,\ y_k,\ z_k + h_k)$$

for each candidate point p_i **do**
    determine relevant projection axes
    project p_i along the corresponding directions
    stop projection when hitting nearest item or container wall
    add resulting EP to `3DEPL`
end for
return updated `3DEPL`

The paper introduces the *Extreme Points First Fit Decreasing* (EP–FFD) heuristic, a packing method built around the notion of *extreme points* (EPs). The heuristic first sorts the items using an externally specified ordering rule and then places them sequentially into the lowest feasible (x, y, z) position

among the currently available EPs. The authors report a time complexity of $O(n^3)$, reflecting the cost of evaluating feasibility and updating the EP set after each placement.

Although the concept of extreme points is foundational, the original work omits several mechanisms essential for practical performance. Notably, it does not describe how to *prune* or *cull* dominated or irrelevant EPs, despite illustrations in the paper suggesting that some form of culling is implicitly applied - Figure 3.1b. Without such pruning, the EP set can grow rapidly, leading to unnecessary collision checks and increased computational cost.

Subsequent research introduced important refinements. Other works generalized the EP framework beyond cuboid geometries, handling arbitrary convex shapes and integrating industrial constraints such as stackability, weight limits, and load-bearing checks.[GTMP22] Heßler et al. (2024) extended EP-based heuristics by incorporating efficient space subdivision for faster collision detection [HHW24].

Overall, the extreme point paradigm remains a widely used candidate–placement baseline across many state-of-the-art nesting and three-dimensional bin packing algorithms.

Maximal empty spaces. While EPs represent point locations, the EMS heuristic maintains a set of rectangular *volumes*—specifically, axis-aligned boxes representing empty regions within the container. **Parreño et al. (2008)** serves as a good introduction maximal-space representation for 3D bin packing problems. Their work outlines an algorithm which generates and maintains empty maximal spaces as boxes are placed.

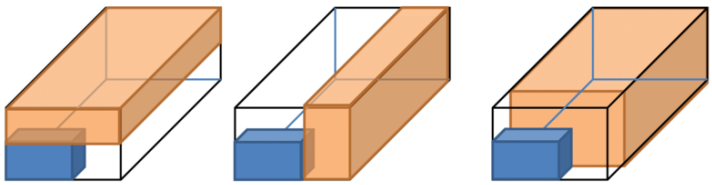


Figure 3.2: Block (blue) creates new empty maximal-spaces (orange)[LZZ14].

An empty maximal space in 3D is defined by its corner position (x, y, z) and dimensions (w, d, h), representing the largest axis-aligned rectangular void at that location. The EMS set is dynamically updated after each placement using *difference operations*: when an item is placed, each EMS that intersects the item's volume is subdivided into up to six new maximal spaces corresponding

to the six axis-aligned slices (left, right, front, back, bottom, top) remaining after the intersection. To prevent combinatorial explosion, EMS that are fully contained within other EMS are eliminated through *dominance pruning*—if space A is entirely within space B, A is discarded since any item fitting in A could also be placed in B at a potentially better position. Extracted from the paper, following algorithm describes Update of Maximal empty spaces after placing a block.[PÁVTO08]

Algorithm 2 Update of maximal-spaces list L after placing a block in S^*

Require:
L – current list of empty maximal-spaces
$S^* \in L$ – maximal-space where the block B has just been packed
B – placed block (axis-aligned rectangular block of boxes)
Ensure:
Updated list L of empty maximal-spaces

Step 3.1: Replace S^* by new spaces created inside it
if B exactly fills S^* **then**
    Remove S^* from L
else
    Remove S^* from L
    Compute the empty maximal-spaces created inside S^* around B ▷ geometric decomposition of $S^* \setminus B$
    Insert all newly created maximal-spaces into L
end if

Step 3.2: Reduce other maximal-spaces intersecting B
for all $S \in L$ such that S intersects B **do**
    Compute $S \setminus B$
    if $S \setminus B = \emptyset$ **then**
        Remove S from L
    else
        Replace S by the maximal-spaces obtained from $S \setminus B$ ▷ split into one or more smaller maximal-spaces
    end if
end for

Step 3.3: Remove contained (non-maximal) spaces
for all pairs of spaces (S_i, S_j) in L, $i \neq j$ **do**
    if $S_i \subset S_j$ **then**
        Remove S_i from L
    else if $S_j \subset S_i$ **then**
        Remove S_j from L
    end if
end for
return L

Given This update mechanism, paper proposes a *greedy randomized adaptive search procedure* (GRASP), an offline, geometrical, partially randomized 3D nesting algorithm with local improvement.

At each iteration, the algorithm selects the maximal-space $S^* \in L$ that is lexicographically closest to a container corner, and anchors the next placement at that corner. For all box types fitting into S^*, the method generates feasible *blocks* (columns or layers of identical boxes) and evaluates them using either volume increase or a best-fit criterion. The best-scoring block is packed into S^*.

After placement, the list of maximal-spaces is updated: S^* is replaced by the new empty regions created around the block; any other maximal-spaces intersecting the block are reduced; and dominated spaces (those contained within others) are removed.

Fanslau and Bortfeldt (2010) present a tree search algorithm using EMS for the container loading problem with weak heterogeneity (items with similar dimensions). They demonstrate that maintaining EMS sorted by volume and applying aggressive pruning—keeping only the top-k largest spaces—significantly improves computational efficiency without sacrificing solution quality.[FB10] Their experiments show that retaining the 1000 largest EMS achieves near-optimal performance while drastically reducing the candidate set size, a finding that directly influenced practical implementations.

Gonçalves and Resende (2013) [GR13] proposes a biased random-key genetic algorithm for 3D bin packing (BRKGA). Their approach combines EMS-based placement with evolutionary item ordering, achieving state-of-the-art results on benchmark instances. They show that EMS-based heuristics naturally encode both spatial efficiency (preferring placements in large voids to minimize fragmentation) and stability (bottom-left placement bias ensures support from below).

Xiong et al. (2024) introduce a generalizable online 3D Bin Packing approach(GOPT), a Transformer-based deep reinforcement learning method for online 3D bin packing in real setting using robotic arm. Their approach represents free space using a compact set of rasterized EMSs extracted from heightmaps, enabling a fixed action space independent of bin size. A Packing Transformer models interactions between items and EMSs via attention, producing placement policies that generalize across bins of different dimensions.[XGP+24].

In summary, EMS-based methods provide a geometrically expressive and computationally tractable framework for representing free volume in 3D packing. Unlike EP heuristics, which enumerate pointwise placements, EMS encodes full empty regions and therefore supports richer operations such as dominance pruning or more accurate feature extraction.

This has made EMS a central component in many high-performing solvers, from classical GRASP-based approaches to evolutionary and genetic metaheuristics, and even modern transformer-driven reinforcement learning systems. Despite methodological differences, these works highlight the same principle: maintaining an accurate, pruned set of maximal empty spaces turns the complex 3D geometry of nesting into a structured action space that can be exploited effectively by both heuristic and learning-based decision models.

3.1.2 Feature extraction

Heightmap.

3.2 Neural Networks

3.2.1 Types of networks

Convolutional neural network.

Graph NN.

3.2.2 Transformer NN

Attention mechanism.

3.2.3 Single use network

3.2.4 Deep reinforcement learning

3.3 Neural Architecture Search

Neurogenesis and Efficient Neural Architecture Search (ENAS) automate the design of neural network architectures through evolutionary algorithms. This section reviews key developments in this field and their relevance to discrete optimization problems.

3.3.1 Overview of ENAS

Elsken et al. (2019) [**EMH19b**] provide a comprehensive survey of neural architecture search methods, categorizing approaches into reinforcement learning-based, gradient-based, and evolutionary methods. They identify evolutionary approaches as particularly well-suited for black-box, multi-objective optimization where gradients are unavailable or unreliable. This flexibility makes ENAS attractive for discrete combinatorial problems like bin packing.

White et al. (2023) [**WSS+23**] analyze insights from 1000 NAS papers, distilling best practices for search space design, performance prediction, and computational efficiency. They emphasize that compact, well-designed search spaces with domain-specific inductive biases outperform unconstrained searches. For nesting problems, this suggests encoding geometric and packing-specific features directly into the architecture search space.

Liu et al. (2020) [**LSX+20**] survey efficient neural architecture search specifically, tracing developments from NEAT (NeuroEvolution of Augmenting Topologies) to modern deep learning contexts. They highlight that evolving topology alongside weights allows discovering novel architectural patterns not easily captured by hand-designed templates. Self-adaptive mutation rates and speciation mechanisms help maintain population diversity and avoid premature convergence.

Real et al. (2017) [**RMS+17**] demonstrate large-scale evolution of

image classifiers, showing that simple evolutionary strategies can discover competitive architectures when given sufficient computational budget. Their work establishes that tournament selection, mutation-only evolution, and aging mechanisms (removing old individuals) form an effective baseline for ENAS.

Booysen and Bosman (2024) [BB24] apply multi-objective efficient neural architecture search to recurrent neural networks, balancing accuracy, model size, and inference time. They use Pareto-based selection (non-dominated sorting) to maintain a diverse set of trade-off solutions. This approach is directly applicable to bin packing, where we must balance packing quality, runtime, and constraint violations.

Takahashi and Kita (2001) [TK01] introduce crossover operators using independent component analysis for real-coded genetic algorithms. Their BLX-α blending method for continuous genes provides effective recombination for hyperparameters like learning rate, dropout, and weight decay, which appear in our genome encoding

3.3.2 Genome represenation

3.4 Gaps in literature

todo: maximal spaces, justify sparse reward enviroment, justify placing block one by one, NNs in nesting, online vs offline, transformer for nesting

2D, 3D online, offline rasterisation, the one where they all bunch up and expand convex concave integration into different problems interesting points, using NN to find them different heuristics used

.

TODO: DQN, transformers, deep reinforcement learning, double dqn, set transformers, maximal empty spaces, single problem nn, feature extraction, heightmaps, graph neural networks, anns, enas, hyperparameters, nn to do nesting, interesting points

Neurogenesis, ENAS different proposed layers, functions description of representations of nn used in literature graph of architecture neural structure vs hyperparameters, which ones

Chapter 4

Proposed method

stuff from research, make some graphs, callback to problem definition in ch2

Chapter 5

Implementation

Chapter 6

Experiments

Chapter 7

Conclusions

Appendix A

Bibliography

[ARCO22] Sara Ali, António Galrão Ramos, Maria Antónia Carravilla, and José Fernando Oliveira, *On-line three-dimensional packing problems: A review of off-line and on-line solution approaches*, Computers & Industrial Engineering **168** (2022), 108122.

[BB24] Reinhard Booysen and Anna Sergeevna Bosman, *Multi-objective evolutionary neural architecture search for recurrent neural networks*, Neural Processing Letters **56** (2024), no. 4.

[BTC23] Guillem Bonet Filella, Alessio Trivella, and Francesco Corman, *Modeling soft unloading constraints in the multi-drop container loading problem*, European Journal of Operational Research **308** (2023), no. 1, 336–352.

[CPT08] Teodor Gabriel Crainic, Guido Perboli, and Roberto Tadei, *Extreme point-based heuristics for three-dimensional bin packing*, INFORMS J. Comput. **20** (2008), 368–384.

[dNAJ21] Oliviana Xavier do Nascimento, Thiago Alves de Queiroz, and Leonardo Junqueira, *Practical constraints in the container loading problem: Comprehensive formulations and exact algorithm*, Computers & Operations Research **128** (2021), 105186.

[EMH19a] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter, *Neural architecture search: A survey*, Journal of Machine Learning Research **20** (2019), no. 55, 1–21, Editor: Sebastian Nowozin.

[EMH19b] ———, *Neural architecture search: A survey*, 2019.

[FB10] Tobias Fanslau and Andreas Bortfeldt, *A tree search algorithm for solving the container loading problem*, INFORMS Journal on Computing **22** (2010), no. 2, 222–235.

[GOGL16] A. Galrão Ramos, José F. Oliveira, José F. Gonçalves, and Manuel P. Lopes, *A container loading algorithm with static mechanical equilibrium stability constraints*, Transportation Research Part B: Methodological **91** (2016), 565–581.

[GR13] José Fernando Gonçalves and Mauricio G.C. Resende, *A biased random key genetic algorithm for 2d and 3d bin packing problems*, International Journal of Production Economics **145** (2013), no. 2, 500–510.

[GTMP22] Mikele Gajda, Alessio Trivella, Renata Mansini, and David Pisinger, *An optimization approach for a complex real-life container loading problem*, Omega **107** (2022), 102559.

[HHW24] Katrin Heßler, Timo Hintsch, and Lukas Wienkamp, *A fast optimization approach for a complex real-life 3d multiple bin size bin packing problem*, 2024.

[LSX+20] Yuqiao Liu, Yanan Sun, Bing Xue, Mengjie Zhang, and Gary G. Yen, *A survey on evolutionary neural architecture search*, CoRR **abs/2008.10937** (2020).

[LZZ14] Xueping Li, Zhaoxia Zhao, and Kaike Zhang, *A genetic algorithm for the three-dimensional bin packing problem with heterogeneous bins*, IIE Annual Conference and Expo 2014 (2014).

[MKS+13] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller, *Playing atari with deep reinforcement learning*, 2013.

[MSIB20] Nina Mazyavkina, Sergey Sviridov, Sergei Ivanov, and Evgeny Burnaev, *Reinforcement learning for combinatorial optimization: A survey*, 2020.

[PÁVTO08] F. Parreño, R. Álvarez-Valdés, J. M. Tamarit, and J. F. Oliveira, *A maximal-space algorithm for the container loading problem*, INFORMS Journal on Computing **20** (2008), no. 3, 412–422.

[RMS+17] Esteban Real, Sherry Moore, Andrew Selle, Saurabh Saxena, Yutaka Leon Suematsu, Jie Tan, Quoc Le, and Alex Kurakin, *Large-scale evolution of image classifiers*, 2017.

[SB18] Richard S. Sutton and Andrew G. Barto, *Reinforcement learning: An introduction*, second ed., The MIT Press, 2018.

[TK01] M. Takahashi and H. Kita, *A crossover operator using independent component analysis for real-coded genetic algorithms*, Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546), vol. 1, 2001, pp. 643–649 vol. 1.

[WHS07] Gerhard Wäscher, Heike Haußner, and Holger Schumann, *An improved typology of cutting and packing problems*, European Journal of Operational Research **183** (2007), no. 3, 1109–1130.

[WSS+23] Colin White, Mahmoud Safari, Rhea Sukthanker, Binxin Ru, Thomas Elsken, Arber Zela, Debadeepta Dey, and Frank Hutter, *Neural architecture search: Insights from 1000 papers*, 2023.

[XGP+24] Heng Xiong, Changrong Guo, Jian Peng, Kai Ding, Wenjie Chen, Xuchong Qiu, Long Bai, and Jianfeng Xu, *Gopt: Generalizable online 3d bin packing via transformer-based deep reinforcement learning*, IEEE Robotics and Automation Letters **9** (2024), no. 11, 10335–10342.

Katedra: matematiky Akademický rok: 2008/2009

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

Pro:	Tomáš Hejda
Obor:	Matematické inženýrství
Zaměření:	Matematické modelování
Název práce:	Spřátelené morfismy na sturmovských slovech / Amicable Morphisms on Sturmian Words

Osnova:

1. Seznamte se se základními pojmy a větami z teorie symbolických dynamických systémů.
2. Udělejte rešerši poznatků o sturmovských slovech: přehled ekvivalentních definic sturmovských slov, popis morfismů zachovávajících sturmovská slova, popis standardních párů slov.
3. Zkoumejte vlastnosti párů spřátelených sturmovských morfismů, pokuste se popsat jejich generování a počty v závislosti na tvaru jejich matice.

Doporučená literatura:

1. M. Lothair, Algebraic Combinatorics on Words, Encyclopedia of Math. and its Applic., Cambridge University Press, 1990
2. J. Berstel, Sturmian and episturmian words (a survey of some recent result results), in: S. Bozapalidis, G. Rahonis (eds), Conference on Algebraic Informatics, Thessaloniki, Lecture Notes Comput. Sci. 4728 (2007), 23-47.
3. P. Ambrož, Z. Masáková, E. Pelantová, Morphisms fixing a 3iet words, preprint DI (2008)

Vedoucí bakalářské práce:	Prof. Ing. Edita Pelantová, CSc.
Adresa pracoviště:	Fakulta Jaderná a fyzikálně inženýrská Trojanova 13 / 106 Praha 2

Konzultant:

Datum zadání bakalářské práce:	15.10.2008
Termín odevzdání bakalářské práce:	**7.7.2009**

V Praze dne 17.3.2009

....

Vedoucí katedry Děkan